Document Number: P0120R0
Date: 2015-09-25
Author: Anthony Williams
Just Software Solutions Ltd

P0120R0: `constexpr` `union`s and common initial sequences

When prototyping my `variant` implementation, I found that the most space-efficient way of storing the values and discriminator was to create a union of `struct`s where each `struct` had the discriminator as the first field, and the data as the second field, rather than having a `struct` holding the discriminator and a `union` of the stored types. e.g. for `variant<A,B>` the storage was equivalent to:

```
struct wrapped_A{
    char discriminator;
    A data;
};
struct wrapped_B{
    char discriminator;
    B data;
};
union storage{
    wrapped_A a;
    wrapped_B b;
} u;
```

instead of

```
struct storage{
    char discriminator;
    union {
        A a;
        B b;
    } u;
};
```

By 9.2 [class.mem] p18, it is OK to access the common initial sequence (the discriminator) of the union members, whichever member is active, so I could always read `u.a.discriminator` in order to check the stored type.

This can be more space efficient than the alternative if the alignment on the largest field is less-strict than the alignment on another field. e.g. if `A` is a `struct` holding 5 `int`s, whereas `B` is a pointer, if `int`s are 4-byte aligned, but pointers are 8-byte aligned.

Unfortunately, when I tried to make this `constexpr` correct, it didn't work: all the compilers I had complained that I was accessing a non-active `union` member if anything other than the first member was active.

In the core reflector message 28016, Richard Smith confirmed that this is correct:

> Per [expr.const]/2.8, "A conditional-expression e is a core constant expression unless the evaluation of e, following the rules of the abstract machine (1.9), would evaluate one of the following expressions: [...] -- an lvalue-to-rvalue conversion (4.1) or modification (5.18, 5.2.6, 5.3.2) that is applied to a glvalue that refers to a non-active member of a union or a subobject thereof;"

I think this is unfortunate, and don't see any need for this restriction. It is already allowed at runtime, so why not in `constexpr` expressions?

By restricting the use of this in `constexpr` expressions people are forced to choose between being space-efficient and `constexpr`-compatible, which seems a poor choice to have to make.

Proposal

Modify [expr.const] paragraph 2 bullet 8 as follows:

> — an lvalue-to-rvalue conversion (4.1) or modification (5.17, 5.2.6, 5.3.2) that is applied to a glvalue that refers to a non-active member of a union or a subobject thereof except that if a standard-layout union contains two or more standard-layout structs that share a common initial sequence, and if the standard-layout union object currently contains one of these standard-layout structs, it is permitted to inspect the common initial part of any of them (see 9.2 [class.mem]);